MODULE

27

Docker, CI/CD & Azure

From ”works on my laptop” to a tested, containerised service running in the cloud.

YOU WILL LEARN

Production-ready tour	`01_production_ready_tour.py`
The service	`app/main.py`
Dockerfile	`Dockerfile`
docker compose	`docker-compose.yml`
CI/CD pipeline	`github-actions/ci-cd.yml`
Azure deploy	`azure/deploy.sh`
Interview Q&A	`INTERVIEW.md`

WHY DEPLOYMENT

”It works on my machine”

THE PROBLEM

Your API runs on your laptop. On the server: a different Python, a missing library, no database, secrets in a file nobody has, and deploying means SSH and crossed fingers.

THE IDEA

Containers package the app with everything it needs. CI/CD tests and ships every change automatically.

STEP BY STEP 12-FACTOR

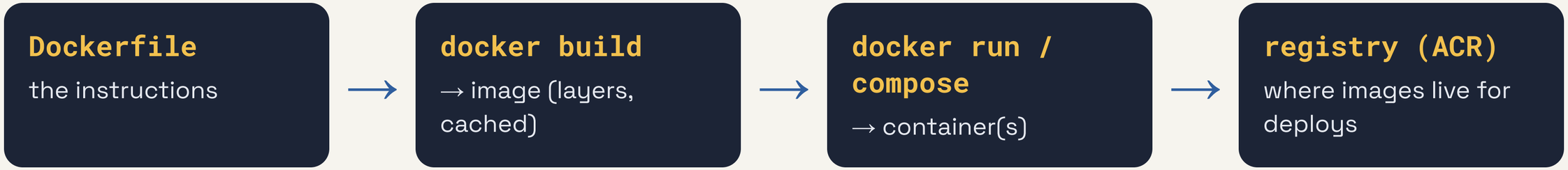
What "production-ready" means

PRINCIPLE	IN PRACTICE
Config in the environment	`DATABASE_URL`, secrets from env vars / Key Vault, never in git
Logs to stdout	JSON lines; the platform collects them
Stateless processes	no files or sessions on local disk; use DB, Redis, blob storage
Health checks	`/health/live` (restart me) and `/health/ready` (send me traffic)
Same build everywhere	one image promoted from test to production
Graceful shutdown	finish requests on SIGTERM

PICTURE IT DOCKER

Image vs container

📦 An **image** is a frozen recipe-plus-ingredients (OS layer, Python, your code). A **container** is a running copy of it. Build once, run the same image on your laptop, in CI and in Azure.



COMMON MISTAKE DOCKERFILE

The slow, unsafe Dockerfile

✗ WRONG

```
FROM python:latest
COPY . .                          # code first
RUN pip install -r requirements.txt
ENV SECRET_KEY=abc123             # baked-in secret
CMD uvicorn app.main:app --reload
```

latest changes silently; any code change re-installs every library; secrets end up in the image; runs as **root** with the dev reloader.

✓ RIGHT

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app ./app                    # code LAST: layers cache
RUN useradd -m appuser
USER appuser
HEALTHCHECK CMD ["python", "healthcheck.py"]  # no curl in slim
CMD ["gunicorn", "app.main:app", "-b", "0.0.0.0:8000",
     "-k", "uvicorn.workers.UvicornWorker"]
```

Pinned slim base, deps before code, non-root, health check, secrets at **runtime** via env.

TRY IT DOCKER COMPOSE

The whole stack with one command

```yaml
services:
  api:
    build: .
    ports: ["8000:8000"]
    environment:
      DATABASE_URL: postgresql://app:app@db:5432/app
      REDIS_URL: redis://cache:6379/0
    depends_on: [db, cache]
  db:
    image: postgres:16
    environment: {POSTGRES_USER: app, POSTGRES_PASSWORD: app}
    volumes: ["pgdata:/var/lib/postgresql/data"]
  cache:
    image: redis:7
volumes:
  pgdata:
```

1. **docker compose up**: API + Postgres + Redis.
2. Services find each other by **name** (db, cache).
3. A **volume** keeps the DB data.

PICTURE IT CI/CD

Nothing ships unless the tests pass

🚦 Every push runs the pipeline. A red step stops the line. Only green builds on **main** are deployed, tagged with the **commit SHA** so you always know what's running and can roll back.



TRY IT CI/CD

A GitHub Actions pipeline

```yaml
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: {python-version: "3.12"}
      - run: pip install -r requirements.txt
      - run: ruff check . && mypy app
      - run: pytest -q
  deploy:
    needs: test
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
      - uses: azure/login@v2          # OIDC, no stored password
      - run: az acr build -t api:${{ github.sha }} .
      - run: az containerapp update --image ...:${{ github.sha }}
```

1. **needs: test**: deploy only after green tests.
2. **OIDC** login: no long-lived cloud secrets in GitHub.
3. Tag with the **SHA** for traceability and rollback.

IN REAL JOBS DEPLOYMENT

Running services in the cloud

Azure Container Apps

```
HTTPS, autoscaling (to zero),
revisions for rollback, secretref: for secrets
```

Observability

```
structured logs + metrics + traces
(App Insights / OpenTelemetry)
```

Zero-downtime deploys

```
rolling updates + readiness probes
+ backwards-compatible migrations
```

Scale

```
stateless replicas behind a load balancer
DB connection pooling, Redis shared cache
```

INTERVIEW SAY THIS IN THE INTERVIEW

“How do you deploy a Python web app to production?”

- Make it **12-factor**: config/secrets from env, logs to stdout, stateless, health endpoints.
- Package it in a **Docker image**: slim pinned base, deps before code, non-root, production server (gunicorn/uvicorn workers).
- **CI** runs lint, types and tests on every PR; **CD** builds the image, tags it with the SHA, pushes to a registry and deploys.
- Run migrations as a release step; use rolling deploys with readiness probes.
- Monitor: logs, metrics, alerts, error tracking; keep a rollback path.

> **Follow-up they ask next:** ”Image vs container? What’s a layer cache?” (image = template, container = running instance; each Dockerfile step is a cached layer, so copy requirements before code).

INTERVIEW SAY THIS IN THE INTERVIEW

“What is CI/CD? What goes in a pipeline?”

- **Continuous Integration**: every change is merged often and automatically built and tested.
- **Continuous Delivery/Deployment**: every green build can be (or is) released automatically.
- Typical stages: checkout → install → lint/format → type check → tests (+ coverage) → security scan → build image → deploy to staging → production.
- Branch protection: PRs need green checks and a review.

> **Follow-up they ask next:** ”Liveness vs readiness probe?” (liveness: is the process alive, else restart it; readiness: can it take traffic now, e.g. DB reachable).

RECAP REMEMBER THESE

Module 27 in 5 lines

1. 12-factor: env config, stdout logs, stateless, health checks.
2. Dockerfile: pinned slim base, deps first, non-root, HEALTHCHECK, gunicorn.
3. **docker compose** for the local stack.
4. CI: ruff, mypy, pytest on every PR; CD only from green main, tagged with the SHA.
5. Azure Container Apps: HTTPS, autoscale, revisions, secrets.

PRACTICE NOW

Do exercises/ex27_fix_the_dockerfile.py: turn a dangerous Dockerfile into a production one.